Robustness of Algorithmic Collusion to Information and Timing Frictions in Q-Learning Pricing

Code on GitHub

Lukáš Chudý

June 3, 2026

Abstract

Reinforcement learning pricing algorithms can sustain prices above the competitive level without communicating, creating a practical problem for competition policy built around explicit agreement. This thesis asks whether real-world information and timing frictions make that concern smaller or larger. I add three frictions to the Calvano et al. (2020) Q-learning pricing baseline: observation latency, noisy monitoring, and asynchronous price setting. Collusion is measured by the profit-gain index Δ, where $\Delta = 0$ corresponds to competitive Nash profits and $\Delta = 1$ to joint-monopoly profits. Varying each friction separately produces three distinct responses in Δ: a sharp initial drop for latency, a smooth decline for noise, and a step then plateau for asynchrony. A $2 \times 2 \times 2$ factorial experiment, replicated at $N \in \{250, 500, 1000\}$ sessions, then finds two interaction patterns that survive replication. Latency and noise compound super-multiplicatively, with joint disruption stronger than independent dampening would predict. Asynchrony partially rescues collusion when imposed jointly with the information frictions, reversing the sign of its standalone effect. The implication is cautious: single-friction evidence can understate the algorithmic-collusion concern, because realistic combinations of frictions may offset disruption rather than simply add to it.

1 Introduction

The practical policy concern behind this thesis is not simply that pricing algorithms can raise prices in a laboratory model. It is whether the messy information and timing conditions of real markets make that concern smaller or larger. Algorithmic pricing is already discussed by competition authorities in sectors such as travel, entertainment, retail, and fuel, and empirical work studies its effects in online retail and German petrol markets (OECD, 2025; Brown and MacKay, 2023; Assad et al., 2024). In the German petrol market, for example, Assad et al. (2024) find that markups rose around the time major chains adopted algorithmic pricing software. These markets do not look like frictionless textbook games. Firms observe rivals through booking systems, web scrapers, price feeds, shelf tags, and reporting platforms, all of which can be delayed, noisy, or tied to fixed update schedules. If those frictions already weaken algorithmic collusion, then the policy problem is less urgent than the clean simulation literature suggests. If they can instead preserve or even restore collusion when combined, then the problem is larger than one would infer from studying each friction in isolation.

The legal difficulty is that cartel enforcement is built around agreement. Explicit price fixing is illegal because it raises prices above the competitive level, transfers surplus from consumers to firms, and creates deadweight loss (Government of the Netherlands, 2024). But autonomous algorithmic coordination may not involve the kind of communication, meeting of minds, or explicit agreement that competition law is normally designed to prove. The OECD therefore treats algorithmic collusion as a challenge for traditional concepts of agreement and tacit coordination (OECD, 2017, 2025). Calvano et al. (2020) make the concern concrete: two independent Q-learning algorithms in a repeated pricing game learn prices about 84% of the way from competitive Nash to joint monopoly, without ever communicating. In economic terms the outcome resembles a cartel, but the conduct is generated by trial-and-error learning rather than an explicit agreement. The full literature, including the follow-up simulation studies and the empirical evidence, is reviewed in Section 2.

The natural next question is whether the Calvano result survives once the information environment becomes less clean. This matters both descriptively and for policy. Descriptively, real pricing algorithms do not always see rival prices instantly or perfectly. They often receive stale, rounded, scraped, or otherwise imperfect signals. For policy, these same features are possible indirect levers: a regulator may not be able to prosecute communication-free learning directly, but could still ask whether delayed disclosure, noisy public price feeds, or commitment windows would make algorithmic coordination less likely. Existing papers study some of these channels one at a time. What is less clear is whether they still work when imposed together, or whether one friction can undo the effect of another.

I keep the Calvano et al. (2020) pricing environment fixed and change only the way the algorithms observe or time their decisions. In the baseline, two firms repeatedly choose prices, consumers respond according to logit demand, and each firm is controlled by a Q-learning algorithm that learns from its own profits and the rival's previous price. I measure the outcome using the profit-gain index Δ. This is an economic collusion score: $\Delta = 0$ means profits are at the competitive Nash benchmark, while $\Delta = 1$ means profits are at the joint-monopoly benchmark. A value such as $\Delta = 0.5$ therefore means that the algorithms have recovered about half of the profit gap between competition and joint monopoly, which is already a substantial supracompetitive outcome. The thesis asks how this score changes when three frictions are introduced separately and jointly:

RQ1. How does observation latency (delayed observation of the rival's price) affect collusion?

RQ2. How does noisy monitoring (Gaussian misobservation of the rival's price) affect collusion?

RQ3. How does asynchronous price setting (Maskin–Tirole alternation between movers) affect collusion?

RQ4. Do these frictions interact non-linearly when imposed jointly?

Varying each friction separately gives three different response patterns in the collusion score: latency produces a sharp initial drop, noise produces a smooth decline toward the competitive benchmark, and asynchrony produces a one-time drop followed by a plateau. A $2 \times 2 \times 2$ factorial experiment, replicated at three session counts to confirm robustness, then identifies two interaction patterns. Latency and noise compound: imposing both jointly destroys more collusion than independent dampening would predict, because the two frictions degrade different dimensions of the rival's observation, timing and value. Asynchrony has the opposite interaction. Although it reduces collusion on its own, it partially restores collusion when combined with latency or noise. The policy relevance is therefore cautious rather than prescriptive. The results do not say that a particular regulatory package should be adopted. They show that the case for intervention cannot be assessed by testing one friction at a time, because realistic combinations can amplify some anti-collusion effects while offsetting others.

2 Literature

The starting point is the classical economics of collusion. Green and Porter (1984) study repeated competition where firms can only observe each other's market shares with noise, not their actions directly, and show that prices above the competitive level can be supported in equilibrium, but only by occasional price wars that get triggered when demand looks suspiciously low. The reason is that firms cannot tell whether a rival cheated on the implicit agreement or whether demand simply fell, so any sufficiently bad signal has to trigger a punishment to keep cheating off the table. This is the textbook explanation of why markets like petrol retailing show recurring price war episodes even when the firms involved seem to be coordinating most of the time. Maskin and Tirole (1988) build a different repeated pricing model in which firms take turns setting prices, and find that this alternating move structure can sustain either price cycles (Edgeworth cycles) or stable high prices. The two papers introduce the basic ideas of imperfect monitoring, alternating moves, and coordination on a focal high price, which the recent algorithmic collusion literature reuses.

Calvano et al. (2020) provide the foundational result for this thesis. They place two independent tabular Q-learning agents (Watkins and Dayan, 1992) in a repeated Bertrand pricing game with logit demand and a discrete grid of prices. Each agent keeps a Q-table that records what each action has been worth so far in each state, updates the table after every period using the standard Q-learning rule with ε-greedy exploration, and earns the period's profit as its reward. The two agents never communicate. Yet they reliably converge on prices well above the competitive Nash level, typically reaching about 84% of the way from Nash to joint monopoly. The result holds across a wide range of hyperparameter choices and exploration rules. The strategies the agents end up with also seem to behave like reward punishment schemes. When one agent deviates by undercutting, the other temporarily lowers its price too (a short price war), and once the deviation ends the prices return to the high level. So in welfare terms the outcome looks like a cartel, but without any of the illegal communication that prosecuting a cartel would normally require.

A follow-up literature has started to relax the strong assumptions of the Calvano environment one at a time. Calvano et al. (2021) look at imperfect monitoring through a stochastic demand shock in a Cournot setting and find that algorithmic collusion shrinks but does not disappear at moderate noise levels. Klein (2021) studies sequential price-setting as the base game (rather than as a deviation from the simultaneous one), and finds that Q-learners under alternating moves also converge on supracompetitive prices, sometimes through the kind of cyclic dynamics Edgeworth described. Calvano et al. (2023) then ask whether the high prices Q-learners settle on are sustained by real punishment threats or are just side effects of the rule the algorithm uses to pick actions, and propose off-path deviation tests as a way to tell the two apart. The test manually forces one agent to deviate after convergence and watches whether the other retaliates. Asker et al. (2024) vary the learning algorithm itself rather than the information environment, comparing synchronous and asynchronous Q-learning protocols, and show that the choice of algorithm matters a lot for the prices that emerge.

Those papers also give benchmarks for interpreting the single-friction results in this thesis. The noisy-monitoring result below is consistent with Calvano et al. (2021) and Zhang (2025): imperfect rival information weakens learned collusion, and sufficiently strong noise can push the outcome close to the competitive benchmark. The asynchronous-pricing result is more mixed relative to Klein (2021). Like Klein, I find that alternating moves do not eliminate supracompetitive pricing. Unlike Klein's sequential-pricing environment, however, imposing alternation on the Calvano baseline lowers the collusion score sharply before it settles on a lower plateau. This difference is useful because it suggests that asynchrony has two roles: it can support coordination in a game designed around sequential moves, while also slowing or disrupting learning when added as a friction to an otherwise simultaneous game. The off-path tests follow Calvano et al. (2023) by asking whether the resulting high prices are backed by punishment behaviour rather than mere convergence inertia.

There is also early empirical evidence from real markets. Assad et al. (2024) use station level data on the German retail gasoline market and find that markups rose around the time the major chains adopted algorithmic pricing software. Their difference in differences design uses chain headquarters' adoption decisions as the source of variation. Brown and MacKay (2023) use high-frequency online retail data and document that firms running pricing algorithms at different update frequencies sustain higher equilibrium prices than firms updating at the same speed, a finding that links the asynchronous pricing literature directly with the algorithmic collusion one. Together, these two papers suggest that the Calvano simulation is not just a numerical exercise. It captures something that is already happening in markets where algorithmic pricing is widespread.

On the policy side, OECD (2025) surveys competition policy across the G7 and notes both the underlying problem (current law relies on detecting communication, which algorithmic agents do not need) and the early enforcement responses to specific cases. Zhang (2025) asks whether mandated noise injection on the rival's observed price could be a regulatory tool, and finds that adding enough bounded Laplacian noise can push learned prices back to the competitive level in a single friction setting.

What is missing from all of this is any controlled study of what happens when several frictions are imposed at the same time. The one-friction literature is informative, and the single-friction experiments in this thesis deliberately connect back to it: noise weakens collusion, alternating moves still leave positive collusion, and off-path tests can diagnose whether the learned outcome is strategic. But real markets typically have several of these frictions going on at once. A regulator considering more than one policy instrument needs to know whether the instruments add up, cancel out, or interact in some other way. Combining a delayed disclosure rule with a noise injection rule, for example, could disrupt collusion more than either one alone, less than either alone, or something in between. The simulation literature does not currently answer that. This thesis is an attempt to start, using a single controlled environment in which the three frictions are introduced one at a time and then jointly in a $2 \times 2 \times 2$ factorial.

3 Methodology

3.1 Baseline

The baseline reproduces Calvano et al. (2020). Two firms ($n = 2$) repeatedly choose prices for differentiated products. "Bertrand" here means that price is the strategic choice variable. "Logit demand" means that consumers are more likely to buy from the cheaper firm, but some consumers still prefer one product over the other because products are differentiated. The firms have homogeneous product appeal, $a_1 = a_2 = 2$, horizontal differentiation $\mu = 0.25$ (moderate but meaningful preference heterogeneity), and unit marginal costs.

Prices are not continuous. Each firm chooses from the same fixed menu of $m = 15$ prices in every period. Following Calvano et al. (2020), this menu is built around two familiar one-shot benchmarks: the competitive Bertrand-Nash price, where neither firm wants to change price unilaterally, and the joint-monopoly price, where total industry profit is maximised. For the baseline parameters these are approximately $p^{\mathrm{N}} = 1.473$ and $p^{\mathrm{M}} = 1.925$. The grid then runs from 1.428 to 1.970 in equal steps of about 0.039, so both benchmarks lie inside the action set rather than at its boundary. The exact construction is given in Appendix A.

The economic basics (a_0, a, c, μ, ξ) and the derived grid are therefore fixed inputs to every session of every experiment in this thesis, not random draws. What is randomised across sessions is only the learning side of the simulation: the initial Q-table entries, the initial prices the agents start at, the pseudo-random stream driving exploration, and, in noise cells, the per-period Gaussian draws on the observed rival price.

The agents have one-period memory. At time t, the state is the pair of prices chosen in the previous period, $s_t = (p_{1,t-1}, p_{2,t-1})$. With 15 possible prices for each firm, there are $15^2 = 225$ possible states. Each agent stores a tabular Q-table. A row of this table is a state, a column is a possible action, and an action is a price-grid index $1, \ldots, m$. The entry in the table is not itself a price. It is a real-valued score: the agent's current estimate of the discounted profit from choosing that price index in that state. Formally, agent i stores $Q_i(s, a)$ for $s \in \mathcal{S}$ and $a \in \{1, \ldots, m\}$, and updates it by the standard one-step rule

$$Q_{i,t+1}(s_t, a_{i,t}) = (1 - \alpha)\, Q_{i,t}(s_t, a_{i,t}) + \alpha \Big[\pi_i(a_t) + \delta \max_{a'} Q_{i,t}(s_{t+1}, a')\Big], \tag{1}$$

with learning rate $\alpha = 0.15$ and discount factor $\delta = 0.95$. Action choice follows exponentially decaying ε-greedy exploration ($\beta = 4 \times 10^{-6}$): with probability ε the agent experiments with a random price, and otherwise it chooses the price index with the highest current Q-score.

Iterations are grouped into episodes of length 5,000 purely as a convergence checking interval. The Q-table, greedy policy, and ε all carry over across episode boundaries, so a session is one continuous training run rather than a sequence of independent rollouts. Sessions are drawn with fresh random initial Q and prices, run until the greedy strategy is unchanged for four consecutive episodes (or a 1,000 episode cap), and aggregated as the mean of session-level converged

$$\Delta = \frac{\bar{\pi} - \pi^{\mathrm{N}}}{\pi^{\mathrm{M}} - \pi^{\mathrm{N}}},$$

where $\bar{\pi}$ is the average per-period profit at convergence, π^{N} the static-Bertrand Nash profit, and π^{M} the symmetric joint-monopoly profit. So $\Delta = 0$ corresponds to fully competitive (Nash) prices, $\Delta = 1$ to perfect (monopoly) collusion, and the headline Calvano result is that the agents converge on $\Delta \approx 0.84$. Cross-session standard deviation is reported alongside.

Replication and grid robustness

The simulator is a from-scratch C++17 translation of the Calvano Fortran reference. Replication is verified at the session level. Running both binaries on the same baseline parameters with identical `Ran2` seeds, the aggregated outputs (`A_res.txt` reporting Δ and the converged-strategy summary) agree byte for byte, and across 100 sessions of per-session metadata ($\sim$ 3,300 numerical values) only four floating-point fields differ, each by exactly the printf rounding granularity of 0.01. Both implementations report the same $\Delta = 0.84644$ to five decimals at this seed sequence. Running the C++ binary at the 1,000-session draw used elsewhere in the paper gives $\Delta = 0.848$ (cross-session SD 0.109), within sampling noise of Calvano et al. (2020)'s published representative value of approximately 0.84.

A grid-robustness check at $m \in \{15, 25, 50, 100\}$ confirms the baseline is not an artefact of the coarse 15-point grid. Δ falls modestly with finer grids and asymptotes by $m \sim 50$ at $\Delta \approx 0.71$ (Figure 1). The remaining experiments stay at $m = 15$ to keep state-space size and runtime tractable while still being representative of the full-grid behaviour.

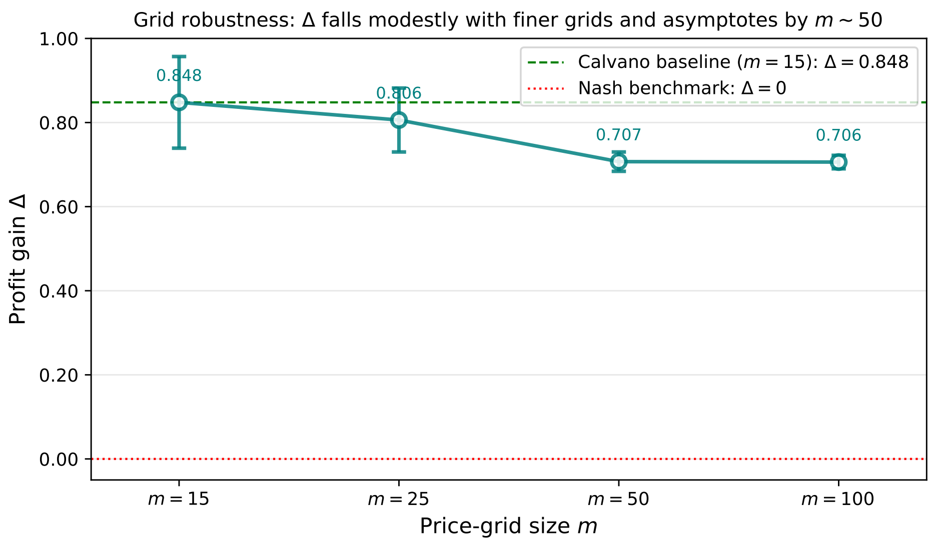

Figure 1: Grid-robustness check at $m \in \{15, 25, 50, 100\}$ (baseline parameters, no frictions). Error bars show cross-session standard deviation. Δ flattens between $m = 50$ and $m = 100$, suggesting the baseline result is not a coarse-grid artefact.

3.2 Three frictions

Each of the three frictions modifies only the information or timing structure facing the agents. The game itself, the action set, and the learning rule are unchanged from the baseline. Setting any friction to its null value reproduces the baseline bit-for-bit, an internal sanity check designed in by construction.

Latency L. Agent i at time t observes the rival's price from period $t-1-L$ rather than $t-1$. The agent's perceived state is $\tilde{s}_t^{(i)} = (p_{i,t-1}, p_{j,t-1-L})$, which differs across agents whenever $L \geq 1$. Profits use the true joint action, so only the agent's view of the rival is degraded. A price-history buffer is a short stored list of each agent's past prices. Under latency L, a per-agent buffer of length $L+1$ is used to retrieve the rival price from L periods ago; it is initialised by repeating the agent's drawn initial price.

The real-world analogue is staleness in cross-firm price observation. Airline ticketing systems propagate fare changes through booking channels and global distribution systems with non-trivial lags. A competitor who lowers fares at 9am may not be visible to you until your scraping or distribution system update at noon. Subscription platform competitors observe each other through monthly catalogue refreshes. The friction parameter L asks how much delay rivals can tolerate before learned coordination breaks.

Noise σ. Each agent observes the rival's last-period price corrupted by an independent Gaussian draw on the price index:

$$\tilde{p}_{j,t-1}^{(i)} \;=\; \operatorname{clamp}\Big(\operatorname{round}\Big(k_{j,t-1} + \sigma\,\varepsilon_t^{(i)}\Big),\ 1,\ m\Big), \qquad \varepsilon_t^{(i)} \sim \mathcal{N}(0,1). \tag{2}$$

Here $k_{j,t-1}$ is the rival's true price index in the previous period. The Gaussian draw first perturbs that index, rounding maps the perturbed value back to an integer grid index, and clamping ensures that the observed index remains inside the feasible range $1,\ldots,m$. For example, an observation rounded below 1 is recorded as 1, and an observation rounded above 15 is recorded as 15. Noise is drawn fresh each period and independently for each agent. Profits are always computed from the true prices, so noise affects only what the learner observes, not the payoff function.

The real-world analogue is data quality friction in cross-firm price intelligence. Firms typically do not observe each other's prices directly. They scrape from competitor websites, buy data feeds, or estimate from indirect signals, and each of these channels introduces noise. Zhang (2025) interprets the same kind of friction as a policy lever, since a regulator could in principle mandate that published prices be reported with bounded noise on the theory that algorithmic collusion fragile to noise can be disrupted by adding it.

Async T. At iteration t, the moving agent's identity depends on the lock-in length:

- if $T=0$, both agents move every period (the simultaneous Bertrand baseline),
- if $T \geq 1$, $\text{mover}(t) \;=\; (\lfloor (t-1)/T \rfloor \bmod 2) + 1$, with the non-mover holding its previous price.

So $T=1$ is strict alternation (Maskin–Tirole, where agent 1 moves on odd t and agent 2 on even t) and $T \geq 2$ is T-period lock-in, where each agent owns the right to revise its price for T consecutive periods, then the other agent takes over. Both agents Q-update each period regardless of who actually moved. The non-mover's action for the period is to repeat its previous price, which is a valid element of its action set, and the rewards both agents observe from the realised joint action are legitimate learning signals.

The real-world example here is asymmetric pricing cadence. Brown and MacKay (2023) document that online retailers update prices at sharply asymmetric frequencies (hourly, daily, and weekly) and show theoretically and in a calibrated counterfactual that this asymmetry sustains prices above the Bertrand-Nash benchmark, with the slower firms charging the higher prices. The mechanism is that the slow firm is effectively committed to its posted price between updates, and the fast firm's algorithm best responds to that posted price rather than undercutting aggressively. In the present model, lock-in may therefore reduce part of the coordination problem: when one firm cannot revise its price, the other firm responds to a known current target rather than to a rival that may change price at the same moment. This mechanism matters again in the combined-friction experiments, where timing structure can partly offset degraded information.

3.3 Combined frictions and the combination design

A unified learner takes (L, σ, T) jointly. Inside each period it (i) computes each agent's perceived state, using the delayed rival-price observation and adding fresh noise when $\sigma > 0$, (ii) determines which agent is allowed to move under T-period lock-in, with the non-mover repeating its previous price, (iii) lets the mover choose a price index by ε-greedy exploration on its perceived state, (iv) computes profits from the true joint prices, (v) updates both agents' Q-tables using their own perceived states, and (vi) advances the price-history buffers and the exploration parameter ε. At all-zero inputs the unified learner reproduces the baseline. At single-friction corners (e.g. $L = 2$, $\sigma = 0$, $T = 0$) it reproduces the corresponding single-friction implementation to numerical precision, which is the built-in cross-validation.

Combination design. The main interaction experiment uses all eight combinations of the three frictions. Each friction has an "off" level (the null value) and an "on" level (a moderate, non-saturating value):

$$L \in \{0, 2\}, \qquad \sigma \in \{0, 1\}, \qquad T \in \{0, 1\}.$$

The eight cells are labelled `F`abc where each digit indicates whether the corresponding friction is on. So `F000` is the baseline, `F100` is "latency only", `F011` is "noise plus async, no latency", and `F111` is "all three on". The on-levels are taken from the single-friction experiments reported in Section 4. They were chosen to make each individual friction visible without pushing the outcome all the way to the competitive benchmark. In those single-friction cells, the collusion score is $\Delta = 0.215$ for latency at $L = 2$, $\Delta = 0.493$ for noise at $\sigma = 1$, and $\Delta = 0.378$ for asynchrony at $T = 1$, compared with the baseline value $\Delta = 0.843$. These values are low enough to show that each friction matters, but high enough that joint effects can still be detected. Each cell is run at $N \in \{250, 500, 1000\}$ sessions to confirm that the interaction patterns are not artefacts of sampling noise.

4 Single-friction results

Each single friction sweep uses 250 sessions per cell. All sessions converged within the 1,000 episode cap. Table 1 reports Δ across the three sweeps. The three shapes that emerge are qualitatively distinct, mapping to three different mechanisms by which information or timing structure interferes with learned collusion.

Table 1: Single friction sweeps. SD is cross-session standard deviation. The $L = 0$, $\sigma = 0$, and $T = 0$ cells all reproduce the baseline ($\Delta = 0.843$) bit-for-bit, which is the internal sanity check.

Latency			**Noise**			**Async**		
L	Δ	SD	σ	Δ	SD	T	Δ	SD
0	0.843	0.114	0.0	0.843	0.114	0	0.843	0.114
1	0.280	0.196	0.5	0.670	0.226	1	0.378	0.108
2	0.215	0.110	1.0	0.493	0.290	2	0.418	0.095
3	0.193	0.102	2.0	0.242	0.262	5	0.424	0.116
5	0.174	0.086	3.0	0.143	0.218	10	0.411	0.108
10	0.158	0.085	5.0	0.013	0.064	20	0.403	0.119
20	0.155	0.085	10.0	−0.003	0.073			

4.1 Latency: cliff plus floor

A single period of delay ($L = 0 \to 1$) erases two-thirds of the collusion. From $L \geq 2$ onward Δ continues to fall but more slowly, settling around a residual of $\Delta \approx 0.16$ for $L \geq 5$ (Figure 2). The cliff at $L = 1$ reflects that the Calvano agents' policies condition on the rival's last-period price, and severing that link breaks reward, punishment, and conditioning at once. The residual at high L is more interesting because it implies that some collusion is sustained without timely monitoring, presumably a mutual high price equilibrium that requires no enforcement. We will see in Section 5 that this residual is destroyed when noise is added, which suggests the mechanism it represents requires the rival side observation to be at least correct (even if stale).

This is consistent with the broader imperfect-monitoring literature in the sense that degrading rival information weakens learned collusion (Calvano et al., 2021; Zhang, 2025). The shape is different, however. Noisy monitoring corrupts the value of the rival signal and produces the smooth decline reported below. Latency leaves the rival price value correct but makes it arrive too late, producing a sharp one-period collapse followed by a residual floor. The comparison suggests that timeliness and accuracy are separate channels rather than interchangeable versions of the same information friction.

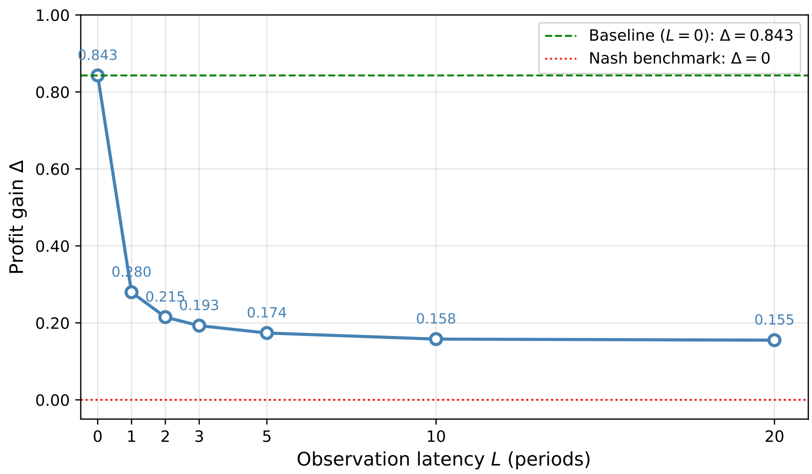

Figure 2: Effect of observation latency L on collusion. The cliff at $L = 1$ accounts for nearly the entire effect, and further latency adds little.

4.2 Noise: smooth decline to Nash

Δ falls continuously from 0.843 at $\sigma = 0$ to indistinguishable from Nash by $\sigma = 5$ (Figure 3). Unlike latency there is no cliff and no residual floor. The mechanism is different. Noise corrupts the rival side state value rather than its arrival time. At high σ the perceived state is close to independent of the rival's actual price, so any policy that conditions on that state encodes no rival relevant information. Both reactive and focal point coordination require the perceived state to contain rival-side information, and noise destroys both.

To translate σ into something concrete, the price grid has 15 slots spanning roughly the Nash to monopoly range, so one grid step corresponds to a few cents of price change in this parameterisation. $\sigma = 1$ then means each agent's observation of the rival is off by about one slot (a few cents) per period, the kind of error you would get from a slightly out of date price feed or a scrape that misses the most recent change. $\sigma = 2$ corresponds to two step errors, noticeable but not catastrophic. By $\sigma = 5$ the noise standard deviation is comparable to half the grid range, the agent essentially cannot read the rival's price, and collusion collapses entirely.

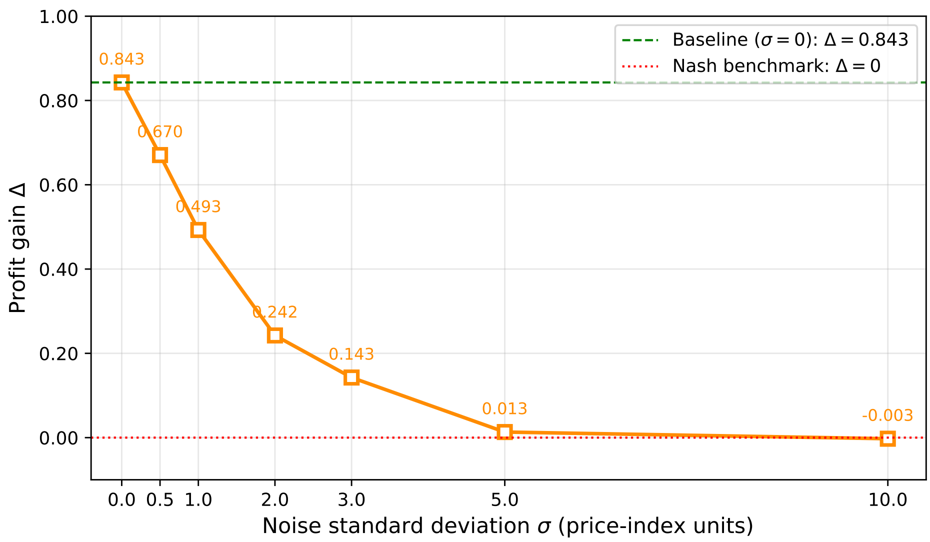

Figure 3: Effect of noisy monitoring σ on collusion. Smooth monotone decline reaching the Nash benchmark by $\sigma = 5$, with no residual floor.

4.3 Async: step then plateau

Strict alternation cuts Δ in half from $T = 0$ to $T = 1$, then plateaus around $\Delta \approx 0.41$ for $T \geq 2$ (Figure 4). The drop seems structural. It depends on whether prices are simultaneous or staggered, not much on how staggered. The result is opposite to what naive Maskin–Tirole reasoning would predict, but consistent with a learning side story. Alternation halves each agent's effective state visitation rate within a fixed iteration budget, so the algorithm has more difficulty discovering the high price equilibria of the simultaneous baseline. A competing strategic explanation is that the equilibrium set of the alternating move game differs from that of the simultaneous one, and the equilibrium that Q-learners select happens to be lower. Distinguishing the two would require a longer horizon experiment.

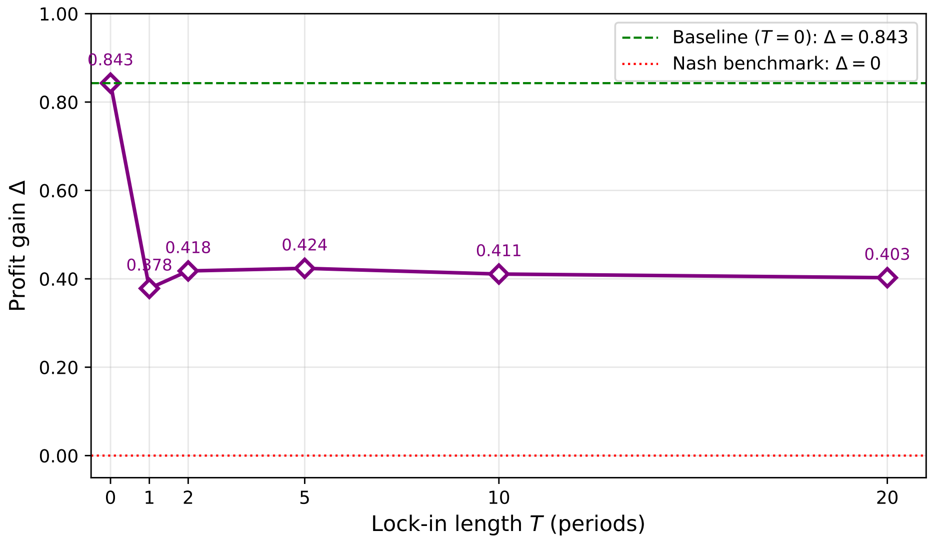

Figure 4: Effect of asynchronous lock in length T on collusion. Step from baseline at $T = 0$ to a plateau at $\Delta \approx 0.41$ for $T \geq 1$.

4.4 Three shapes side by side

Figure 5 places the three sweeps on a shared Δ axis. The qualitative differences are visible at a glance, namely cliff, smooth decline, step then plateau.

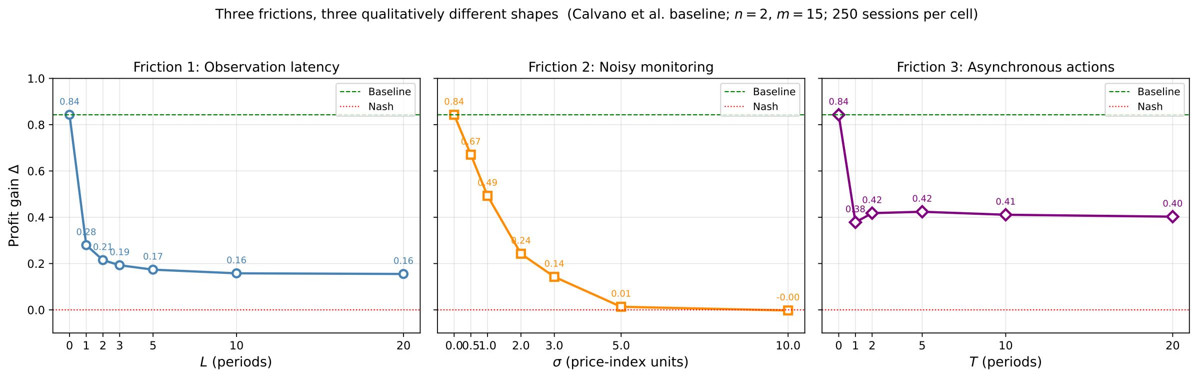

Figure 5: Three frictions, three qualitatively different shapes. The single-friction sweeps interfere with three different parts of the learning machinery. Latency delays signals, noise corrupts them, and asynchrony restructures the game.

5 Combined-friction results

The combined-friction design crosses latency $L \in \{0, 2\}$, noise $\sigma \in \{0, 1\}$, and asynchrony $T \in \{0, 1\}$, giving 8 cells Fabc where each digit records whether the corresponding friction is switched on. Thus `F000` is the no-friction baseline, `F110` has latency and noise but no asynchrony, and `F111` has all three frictions. Each cell is run at $N \in \{250, 500, 1000\}$ sessions to check whether the interaction patterns depend on replication count. Table 2 shows that the largest movement between $N = 250$ and $N = 1000$ is about 0.023 in Δ, while the key cell differences used below are much larger. I therefore report $N = 1000$ as the main estimate (Table 3, Figure 6) and use the smaller-N runs as a stability check.

Table 2: Combined-friction Δ across replication sizes. Cell-level movement between $N = 250$ and $N = 1000$ is uniformly below 0.025, and the qualitative pattern is identical at all three N.

Cell	Δ (N=250)	Δ (N=500)	Δ (N=1000)
`F000`	0.843	0.842	0.848
`F100`	0.215	0.212	0.210
`F010`	0.493	0.496	0.500
`F001`	0.378	0.385	0.383
`F110`	0.063	0.062	0.059
`F101`	0.323	0.343	0.346
`F011`	0.264	0.277	0.280
`F111`	0.551	0.539	0.537

The four single-friction cells (`F000`, `F100`, `F010`, `F001`) reproduce the corresponding single-friction results from Section 4 to numerical precision, confirming that the unified learner correctly composes the three frictions.

5.1 Benchmark predictions

When two or three frictions are imposed together, there is no theoretical benchmark that uniquely determines the joint Δ. I therefore compare the observed multi-friction cells with four reference predictions. These are not structural models; they are diagnostic benchmarks for whether the joint outcome looks additive, multiplicative, saturated by the strongest single friction, or dominated by the least disruptive friction.[1] For each multi-friction cell:

Additive
: $\widehat{\Delta}_{\text{add}} = \max\{0,\ \Delta_0 - \sum_{i\in\mathcal{F}}(\Delta_0 - \Delta_i)\}$. The friction effects add, where each friction subtracts its own reduction from baseline. Floored at zero because Δ cannot go much below the competitive level.

Multiplicative
: $\widehat{\Delta}_{\text{mult}} = \Delta_0 \prod_{i\in\mathcal{F}}(\Delta_i/\Delta_0)$. The friction effects compound, where each friction preserves a fraction of baseline and the joint effect is the product of those fractions.

Minimum
: $\widehat{\Delta}_{\text{min}} = \min_{i\in\mathcal{F}} \Delta_i$. The strongest friction wins and weaker ones add nothing on top. This is what you would expect under perfect saturation.

Maximum
: $\widehat{\Delta}_{\text{max}} = \max_{i\in\mathcal{F}} \Delta_i$. The weakest friction wins and the joint outcome is no worse than the least destructive single friction. This is what you would expect if frictions perfectly substitute for each other.

A worked example. Consider `F110` (latency and noise on, async off). The single-friction reference values are $\Delta_L = 0.210$ (latency only) and $\Delta_\sigma = 0.500$ (noise only), with baseline $\Delta_0 = 0.848$. The four predictions are

- Additive. Latency knocks off $0.848 - 0.210 = 0.638$, noise knocks off $0.848 - 0.500 = 0.348$, total predicted reduction is 0.986, predicted $\Delta = 0.848 - 0.986 = -0.14$, floored at $\widehat{\Delta}_{\text{add}} = 0$.

[1] Notation: Δ_0 is the no-friction baseline, Δ_i is the single-friction value for friction i, and $\mathcal{F}$ is the set of frictions switched on in the cell being predicted. For example, $\mathcal{F} = \{L, \sigma\}$ for `F110` and $\mathcal{F} = \{L, \sigma, T\}$ for `F111`.

- Multiplicative. Latency keeps $0.210/0.848 = 0.248$ of baseline, noise keeps $0.500/0.848 = 0.590$ of baseline, joint kept fraction is $0.248 \times 0.590 = 0.146$, predicted $\widehat{\Delta}_{\text{mult}} = 0.848 \times 0.146 = 0.124$.
- Minimum. $\min(0.210, 0.500) = \widehat{\Delta}_{\text{min}} = 0.210$.
- Maximum. $\max(0.210, 0.500) = \widehat{\Delta}_{\text{max}} = 0.500$.

The four benchmarks span the natural intuitions about combining independent effects. If the observed Δ matches one of them, that tells us how the frictions interact. If it does not match any, that is itself the finding.

5.2 Main combined-friction results

Table 3: Combined-friction design at $N = 1000$. SD is cross-session standard deviation. Single-friction cells (above the line) and multi-friction cells (below) are separated for readability. For multi-friction cells, the benchmark closest to the observed Δ is shown in bold.

Cell	(L, σ, T)	Δ (obs.)	SD	$\widehat{\Delta}_{\text{add}}$	$\widehat{\Delta}_{\text{mult}}$	$\widehat{\Delta}_{\text{min}}$	$\widehat{\Delta}_{\text{max}}$
`F000`	$(0, 0, 0)$	0.848	0.109				
`F100`	$(2, 0, 0)$	0.210	0.122				
`F010`	$(0, 1, 0)$	0.500	0.298				
`F001`	$(0, 0, 1)$	0.383	0.111				
`F110`	$(2, 1, 0)$	0.059	0.080	**0.000**	0.124	0.210	0.500
`F101`	$(2, 0, 1)$	0.346	0.174	0.000	0.094	0.210	**0.383**
`F011`	$(0, 1, 1)$	0.280	0.103	0.034	**0.226**	0.383	0.500
`F111`	$(2, 1, 1)$	0.537	0.135	0.000	0.056	0.210	**0.500**

Figure 6 visualises the same comparison as Table 3. The distance between the black observed points and the benchmark markers is the main evidence for the interaction patterns discussed below.

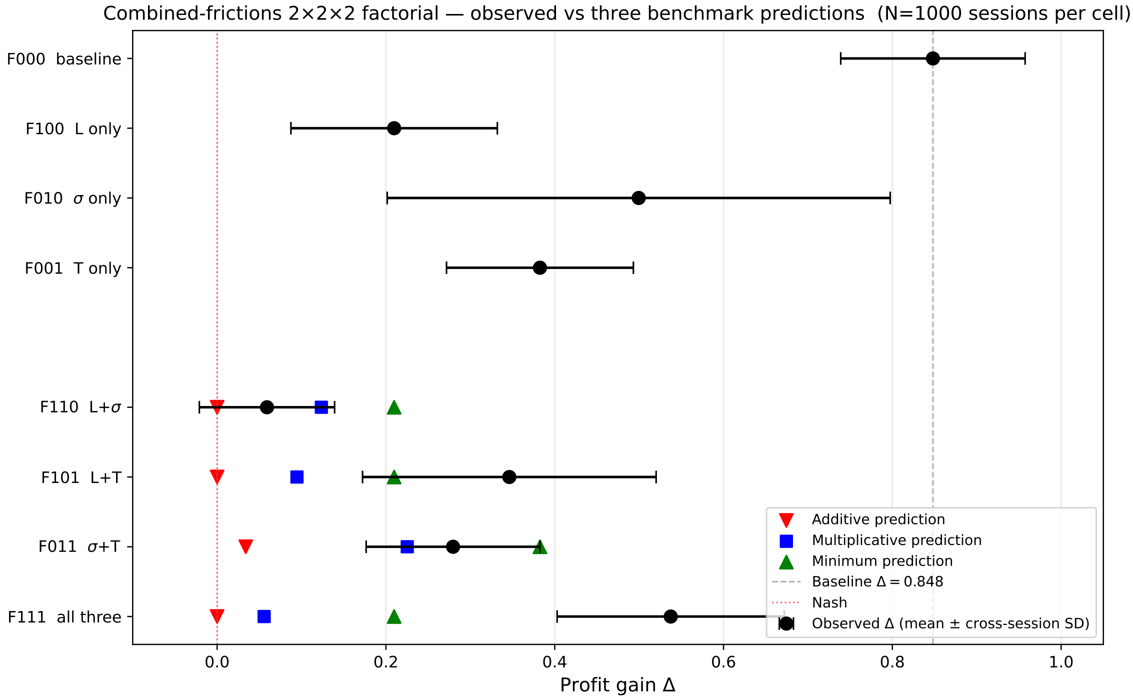

Figure 6: Combined-friction cells at $N = 1000$. Black shows the observed Δ with cross-session SD bars. Coloured markers show the additive (red), multiplicative (blue), and minimum (green) benchmark predictions for each multi-friction cell.

Two interaction patterns emerge.

Latency + noise compound, even more strongly than independent dampening would predict. $\Delta_{\texttt{F110}} = 0.059$ falls below three of the four predictions. It is below the multiplicative (0.124), well below the minimum (0.210), and far below the maximum (0.500). The additive prediction reads as 0 in the table, but only because of the non-negativity floor (the raw additive value is -0.14), so flooring is the only thing keeping it from being even further below the observed 0.059. The substantive comparison is therefore against the multiplicative model, and against that benchmark the observed value is below by a factor of two. When two information degradation frictions are imposed jointly, the joint effect is stronger than the product of their individual effects. We call this super-multiplicative compounding, since it is literally more than the multiplicative model predicts and the two frictions amplify each other rather than just stacking independently. The mechanism is that latency and noise act on different dimensions of the rival side observation. Latency acts on its time of arrival, noise on its value. Either alone leaves some learning signal intact (latency leaves the value correct just stale, noise leaves the timing correct just inaccurate). Combined, neither dimension is reliable, and the residual focal point coordination that survives pure latency is destroyed by the noise component.

Asynchrony raises Δ when it is combined with information frictions. $\Delta_{\texttt{F101}} = 0.346 > \Delta_{\texttt{F100}} = 0.210$, and $\Delta_{\texttt{F111}} = 0.537 > \Delta_{\texttt{F110}} = 0.059$. Adding asynchrony therefore increases Δ relative to otherwise identical cells without asynchrony, even though asynchrony alone reduces Δ from 0.848 to 0.383. I refer to `F101`, `F011`, and `F111` as async-rescue cells in this descriptive sense: asynchrony partially restores collusion that latency or noise would otherwise suppress. The proposed mechanism is that strict alternation imposes a one-period structural commitment on the rival, giving the moving agent a known target. When the rival-side signal is noisy or stale, this commitment partly substitutes for the information the moving agent would otherwise need to infer from an unreliable observation.

Reading these patterns through the four benchmarks, `F101` sits between $\widehat{\Delta}_{\min}$ and $\widehat{\Delta}_{\max}$, and

`F111` sits very close to $\widehat{\Delta}_{\max}$ (0.537 observed against 0.500 from the max benchmark). So once async is in play with one or both other frictions, the joint outcome is approximately as collusive as the least destructive single friction, not the most destructive. That matches a story in which async substitutes for the missing rival side information and effectively neutralises the worse of the two information frictions.

5.3 Effect decomposition of the interactions

The purpose of the decomposition is to express the eight $N = 1000$ cell means in Table 3 as a baseline, three main effects, and four interaction effects. Because there are 8 cells and 8 terms, the design is saturated: the coefficients are an exact re-expression of the cell means, not a prediction model fitted with residual error. The benefit is interpretive. Main effects reproduce the single-friction reductions, while interaction effects measure how far the multi-friction cells depart from a purely additive combination of those reductions.

The decomposition is

$$\bar{\Delta}_c \;=\; \beta_0 + \beta_L L_c + \beta_\sigma \sigma_c + \beta_T T_c + \beta_{L\sigma}(L_c\sigma_c) + \beta_{LT}(L_cT_c) + \beta_{\sigma T}(\sigma_cT_c) + \beta_{L\sigma T}(L_c\sigma_cT_c),$$

where c indexes the eight cells and $L_c, \sigma_c, T_c \in \{0,1\}$ indicate whether each friction is active. There is no ε term because the equation can be solved exactly as $\hat{\beta} = X^{-1}\bar{\boldsymbol{\Delta}}$. Standard errors are still meaningful because each cell mean has sampling uncertainty from 1000 independent sessions. I compute each cell's $\mathrm{SEM}_c^2 = \mathrm{SD}_c^2/N$ and propagate that uncertainty through the linear transformation using $\widehat{\mathrm{Var}}(\hat{\beta}) = X^{-1}\,\mathrm{diag}(\mathrm{SEM}_c^2)\,X^{-\top}$.[2] Results are reported in Table 4.

Table 4: Saturated $2 \times 2 \times 2$ effect decomposition at $N = 1000$ sessions per cell. Standard errors are derived analytically from cross-session SDs. Significance: $^{***}\,p < 0.001$.

Term	Estimate	SE	z	p
Intercept	$+0.848^{***}$	0.003	+246.0	<0.001
L	-0.638^{***}	0.005	−123.3	<0.001
σ	-0.348^{***}	0.010	−34.7	<0.001
T	-0.465^{***}	0.005	−94.5	<0.001
$L \cdot \sigma$	$+0.197^{***}$	0.011	+17.8	<0.001
$L \cdot T$	$+0.601^{***}$	0.008	+72.2	<0.001
$\sigma \cdot T$	$+0.245^{***}$	0.011	+22.0	<0.001
$L \cdot \sigma \cdot T$	$+0.097^{***}$	0.014	+7.0	<0.001
Joint Wald test, all four interaction terms equal zero, $\chi^2(4) = 21\,023$, $p < 10^{-15}$.				

The intercept recovers $\Delta_0 = 0.848$ and the three main effects reproduce the single-friction reductions (−0.638 for latency, −0.348 for noise, −0.465 for async). These are mechanical restatements of the cell means.

The interactions carry the substantive content. The latency-times-async coefficient $\beta_{LT} = +0.601$ is by far the largest. It says that when asynchrony is added to latency, Δ is 0.601 higher than a purely additive calculation would imply. The noise-times-async coefficient $\beta_{\sigma T} = +0.245$ gives the analogous result for noise plus asynchrony, smaller in magnitude because noise is a less destructive single-friction treatment than latency. The three-way coefficient $\beta_{L\sigma T} = +0.097$ indicates that, with all three frictions active, the positive asynchrony interaction is mildly stronger than the two-way terms alone would imply.

[2] Implemented in `numpy` and `scipy.stats`. The resulting standard errors reflect uncertainty in the estimated cell means, not residual variation from an unsaturated regression.

The latency-times-noise coefficient $\beta_{L\sigma} = +0.197$ might look at first like it contradicts the super-multiplicative finding for `F110`, but it does not. The additive prediction for `F110` without the zero floor is -0.138, the observed 0.059 sits above that, and a positive $\beta_{L\sigma}$ is mechanically forced by the floor at zero. Against the multiplicative benchmark, observed 0.059 remains below the predicted 0.124, which is the substantive claim. The regression measures additive deviations, not multiplicative ones, so it cannot adjudicate the multiplicative comparison directly.

The joint Wald test on all four interactions ($\chi^2(4) = 21{,}023$, $p < 10^{-15}$) decisively rejects the additive null. The frictions interact, and the patterns identified in the cell-mean analysis are not sampling artefacts. As a separate robustness check, Table 2 shows that changing the replication count from 250 to 1000 sessions shifts each cell mean by less than 0.025 in Δ; a visual version of this check is reported in Appendix A.

5.4 Generalisation across the parameter space

The combined-friction table tests one specific (L, σ, T) point. To check whether the patterns generalise, I re-run the unified learner on a 5×5 grid of parameter values for each of the three pairs (Latency $\times$ Noise, Latency $\times$ Async, Noise $\times$ Async), with the third friction held at zero. The axis values ($L \in \{0, 1, 2, 3, 5\}$, $\sigma \in \{0, 0.5, 1, 2, 3\}$, $T \in \{0, 1, 2, 5, 10\}$) include the combined-friction table's corner values. This gives a direct consistency check: for example, the $(L = 2, \sigma = 1)$ cell of the Latency$\times$Noise heatmap should match `F110`. The heatmap cells use $N = 250$ sessions rather than $N = 1000$ because the purpose is to show qualitative gradients over 75 pairwise cells, not to estimate every cell to four decimal places. The replication-count check in Table 2 indicates that these point estimates should be read with roughly 0.025 precision in Δ.

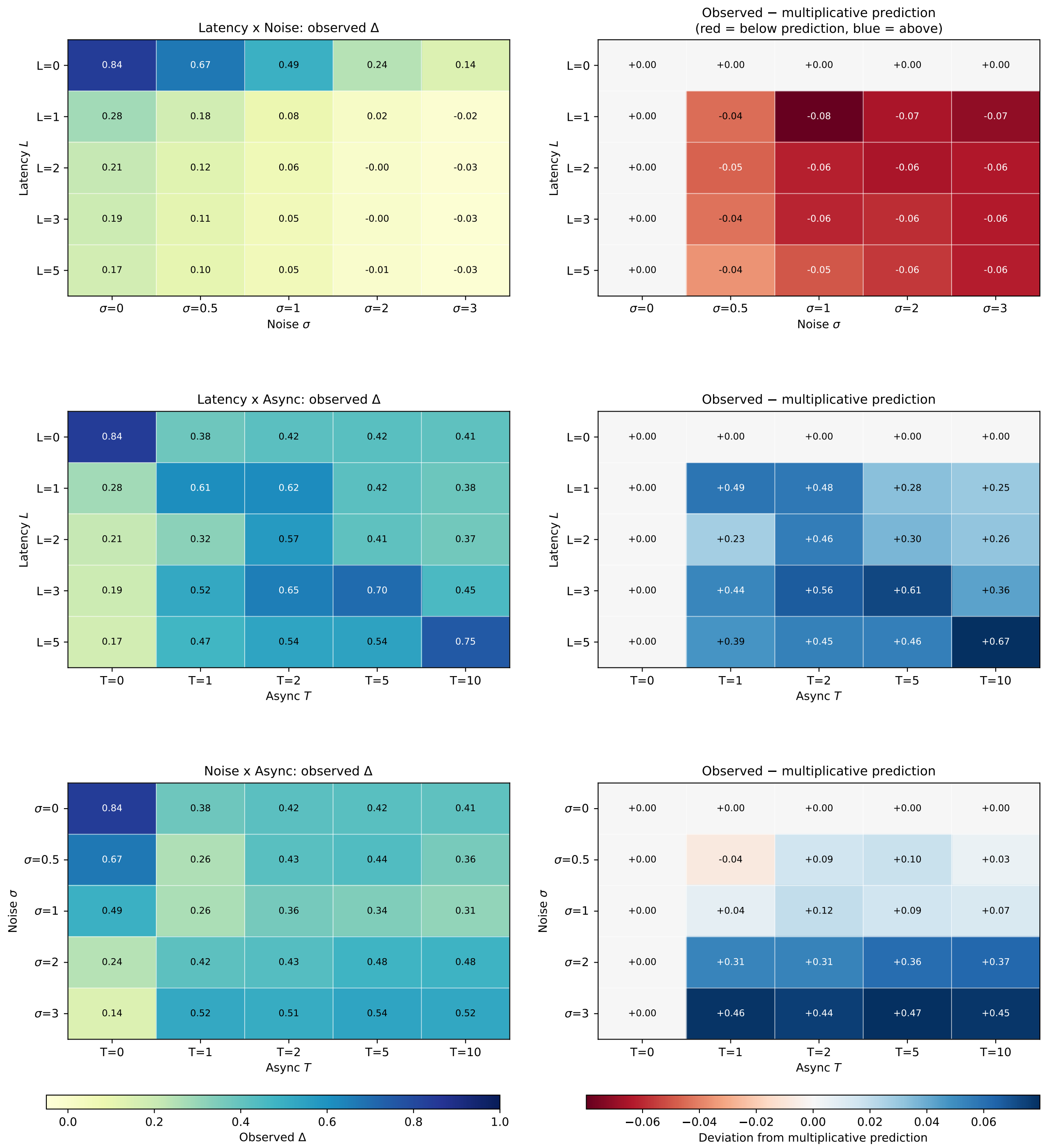

Figure 7: Pairwise heatmaps at 250 sessions per cell. Left column: observed Δ. Right column: observed Δ minus the multiplicative-benchmark prediction (red = below prediction, blue = above). Rows are the three friction pairs, with the third friction held at zero. The single-friction sweeps from Section 4 sit on the zero rows and columns and reproduce the same numbers.

Three patterns confirm that the combined-friction findings generalise.

Latency $\times$ Noise (top row). The right-column heatmap is mostly red across the interior, meaning observed Δ falls below the multiplicative prediction throughout the joint-friction region. The super-multiplicative compounding identified in `F110` is not a one-corner phenomenon. The red region is darkest where both frictions are moderate ($L = 2$–3, $\sigma = 1$–2), at higher noise values the multiplicative prediction is already near zero and the observed value cannot fall much further, so the deviation shrinks.

Latency $\times$ Async (middle row). The right-column heatmap is strongly blue across the interior. Observed Δ sits well above the multiplicative prediction wherever asynchrony is on with non-zero latency. The blue region intensifies along the $T \geq 1$ rows, confirming that the async-rescue pattern identified at `F101` is typical throughout the L $\times$ T plane.

Noise $\times$ Async (bottom row). The same positive asynchrony interaction appears, but it is milder. Asynchrony lifts Δ above the multiplicative prediction whenever both frictions are active, while the magnitude remains smaller than in the L$\times$T case, consistent with $\widehat{\beta}_{\sigma T} < \widehat{\beta}_{LT}$.

Together with the combined-friction corner, the heatmaps show that the two main interaction patterns (super-multiplicative compounding of L and σ; positive asynchrony interactions with information frictions) are not artefacts of the specific parameterisation. They are visible across a continuous range of friction strengths.

5.5 Off-path deviation tests

The interaction analysis tells us what happens at convergence, but not why. Specifically, the interpretation above is that asynchrony partially restores collusion by giving the moving agent a known committed target. An alternative explanation is that the high Δ in the async-rescue cells reflects focal-point lock-in with no reactive punishment behind it, in which case the agents would not punish a defector and the supracompetitive prices would not survive an actual deviation. This is the genuine-versus-spurious distinction of Calvano et al. (2023).

To test this, I extend the unified learner with a playback phase that starts after each session has converged. At playback period 1, agent 1 is forced to deviate, both agents otherwise play their converged greedy strategies, and the Q-tables are frozen. Two deviation types are tested per session: a "Low" deviation (forcing the lowest grid price, the maximum-aggression undercut) and a Bertrand deviation (forcing the static Bertrand best response against agent 2's true current price, the most tempting one-period deviation). The same noise random-number stream is used for both deviation types, so the deviation type is the only difference between them. The playback window was initially set to 30 periods based on the recovery timescales reported in Calvano et al. (2020). This was sufficient for seven of the eight combined-friction cells but left `F111` ambiguous because Δ dropped sharply and did not visibly recover inside the window. The window was therefore extended to 100 periods for all eight combined-friction cells, keeping the panels comparable, and the genuine-versus-spurious diagnostic is taken from the 100-period results. The pairwise heatmap cells retain the original 30-period window because their retaliation dynamics are not at the diagnostic margin. Figure 8 plots the Δ trajectories; statements about agent 2's price response are computed from the same playback files as average changes in agent 2's price-grid index.

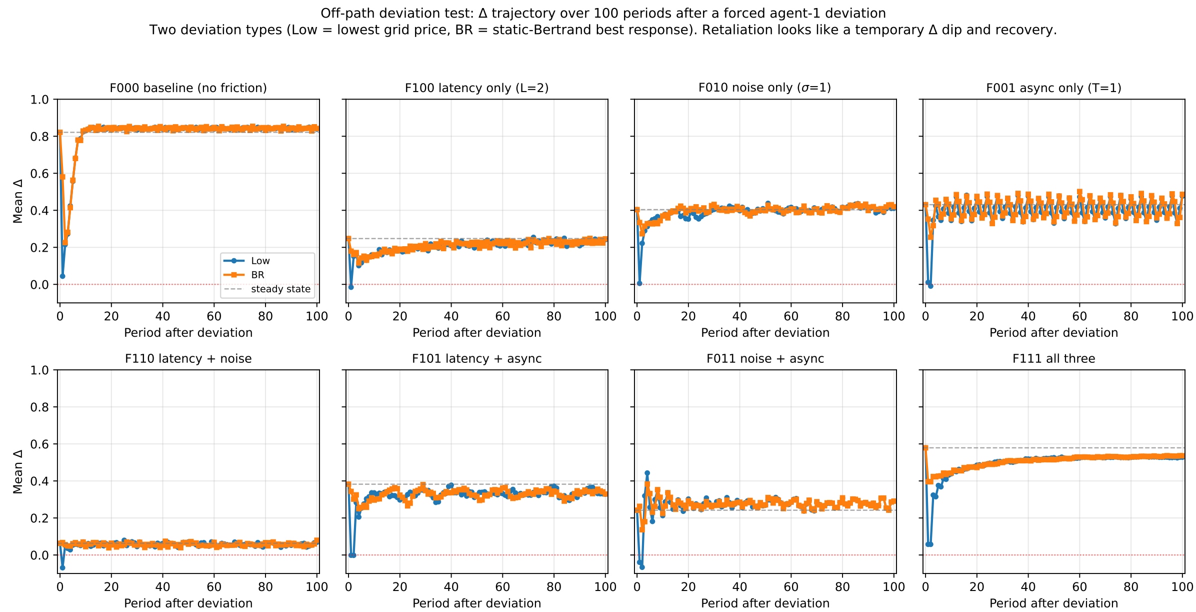

Figure 8: Off-path deviation test on the eight combined-friction cells. Each panel shows the mean Δ across sessions over 100 periods after a forced agent-1 deviation, separately for the Low (blue) and BR (orange) deviation types. The dashed grey line is the pre-shock steady state Δ. A drop-and-recover pattern indicates genuine reward-punishment behaviour, while an immediate snap-back would indicate a spurious focal-point equilibrium. The window was extended from an initial 30 periods to 100 after the first pass left `F111` ambiguous.

The combined-friction panels (Figure 8) show a clear gradient of retaliation strength.

Baseline (`F000`). Both deviation types trigger a deep Δ drop (to near Nash for the Low deviation, to ≈ 0.23 for BR), followed by recovery within roughly seven periods to the pre-shock $\Delta \approx 0.82$. Agent 2's price falls by approximately six grid steps during the punishment phase. This is the reward punishment pattern of Calvano et al. (2020), recovered here as a control.

Single-friction cells (`F100`, `F010`, `F001`). Retaliation is visibly weaker but present. Agent 2 lowers its price by one grid step or so on average, and recovery takes longer (over 20 periods for latency only). The reward-punishment mechanism survives each friction, but in attenuated form.

Async-rescue cells (`F101`, `F011`, `F111`). These are the direct tests of the mechanism proposed in Section 5.2. `F101` shows clear retaliation, as Δ drops to roughly Nash after both deviation types, with recovery in 9–15 periods. Agent 2's price drops by about 2 grid steps on average. This indicates that the high Δ in `F101` is supported by learned punishment behaviour rather than by a passive focal point. `F011` shows a similar but smaller response, with recovery in 5–6 periods. `F111` is the slowest case. The initial 30-period playback made the Δ dynamics ambiguous because the post-deviation drop had not yet recovered within the observed window. Extending the playback to 100 periods resolves the picture: Δ does recover, but much more slowly than in the lower-friction cells. After the Low deviation it climbs steadily from the post-shock minimum of 0.06 back through 0.50 around period 30 and reaches the recovery threshold (within 0.05 of the pre-shock 0.58) at period 74. The BR deviation recovers slightly faster, at period 59. The agents therefore re-establish the converged greedy joint action, but the combined noisy and lagged observation makes coordination roughly an order of magnitude slower than in cells with fewer information frictions. The rescue mechanism in `F111` is genuine in the same sense as in `F101`, but it operates on a much longer dynamic timescale.

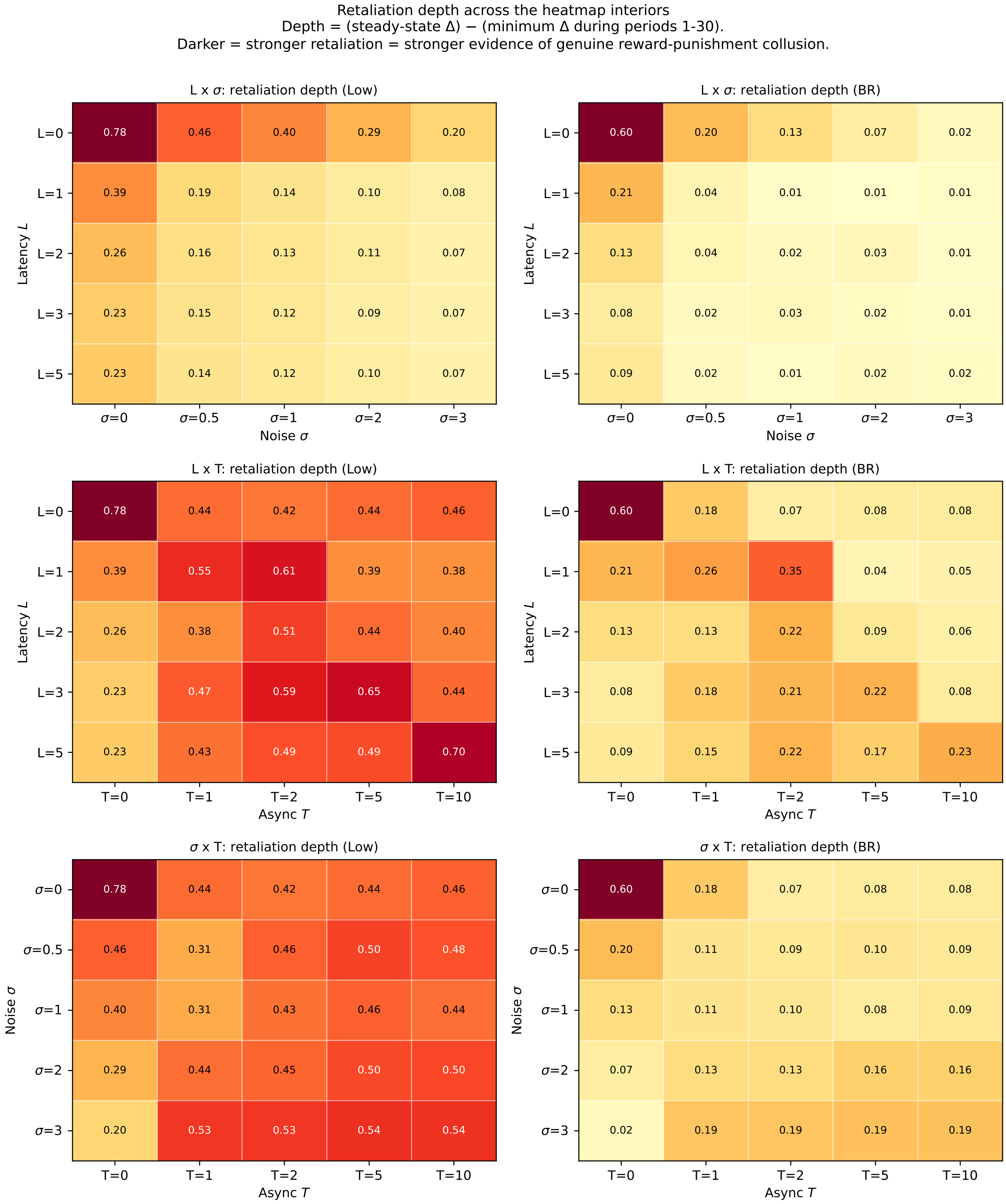

Figure 9: Retaliation depth across the three pairwise heatmaps. Depth is the pre-shock Δ minus the minimum Δ over the 30 post-shock periods, separately for the Low and BR deviation types. Low forces the lowest grid price; BR forces the static Bertrand best response. Darker cells indicate stronger retaliation and therefore stronger evidence of genuine punishment.

The retaliation depth heatmaps (Figure 9) extend the genuine versus spurious diagnostic across the parameter space. Retaliation is deepest in the low-friction regions (closer to the baseline corner) and shallowest in the high-friction interior, with the BR deviation generally producing milder dips than the Low deviation. The L$\times$T panel in particular shows that the rescue cells along the $T \geq 1$ rows retain non-trivial retaliation depth, supporting the genuine-collusion reading of the rescue mechanism throughout the heatmap interior, not just at the factorial corner.

The off-path tests support the genuine collusion reading of the async rescue across all rescue cells. In `F101` and `F011` retaliation is sharp and recovery completes within 5–15 periods. In `F111` the same dynamics happen on a much longer timescale (recovery only at periods 59–74), but the agents do eventually recoordinate on the converged greedy joint action. None of the rescue cells shows the focal point signature of a spurious equilibrium (an immediate snap-back to pre-shock Δ with no retaliation). The mechanism story from Section 5.2, that asynchrony's structural commitment substitutes for the missing rival-side information holds up across the full range of friction combinations tested.

6 Discussion

6.1 Three shapes, three mechanisms

The single-friction sweeps map cleanly to three distinct mechanisms by which information or timing structure interferes with learned collusion. Latency delays signals. It destroys reactive coordination at $L = 1$ but leaves a residual focal point coordination that does not require timely monitoring (the $\Delta \approx 0.16$ floor for $L \geq 5$). Noise corrupts signal value rather than its timing. At high σ the perceived state is nearly independent of the rival's actual price, so policies on that state encode no rival relevant information. Both reactive and focal point mechanisms are destroyed, and there is no residual floor. Async restructures the game. Under strict alternation each agent's effective state visitation rate is halved within a fixed iteration budget, and the algorithm has more difficulty discovering the high price equilibria of the simultaneous baseline.

The fact that the three frictions act on three different parts of the learning machinery is what makes their interaction non-trivial. Latency leaves the value of the rival side observation correct (just stale, useful for focal point coordination), noise leaves the timing correct (so an agent could in principle de-noise across many observations, even if the Q-learner does not actually do so), and asynchrony leaves both timing and value of the per-period observation intact but reduces the effective number of distinct learning episodes. These channels are orthogonal in the mechanical sense that each acts on a separate part of the learning system. But mechanical orthogonality is not the same as effect orthogonality. Two frictions can have non-overlapping mechanisms and still produce a joint Δ that is not the sum or product of their single-friction effects, and the factorial shows that all three pairwise combinations interact in exactly this way.

6.2 Latency plus noise, compounding

The factorial's strongest finding is that latency and noise compound super-multiplicatively. $\Delta_{\texttt{F110}} = 0.059$ falls below both the multiplicative (0.124) and minimum (0.210) predictions. The two frictions act on different dimensions of the rival side observation. Latency acts on its time of arrival, noise on its value. Either alone leaves some learning signal intact. Combined, neither timing nor value is reliable, and the residual focal coordination that survives pure latency is destroyed by the noise component. In economic terms, this means information degradation rules behave as policy complements. Combining a delayed disclosure rule with a noise injection rule destroys more collusion than either rule alone, and more than naive analysis would suggest.

6.3 Asynchrony as a positive interaction

The most surprising result is that asynchrony, despite reducing Δ in isolation, partially restores it when imposed jointly with the other frictions. The mechanism we tentatively offer is that strict alternation imposes a one-period structural commitment on the rival, giving the moving agent a known target against which to optimise. When the rival-side signal is itself corrupted or stale, this structural commitment partially substitutes for the missing information. Brown and MacKay (2023) provide empirical support for the same logic outside the simulation setting. In online retail data, firms running pricing algorithms with asymmetric update frequencies sustain higher equilibrium prices than firms updating in lockstep, consistent with the idea that timing structure can substitute for explicit coordination. A competing learning stability explanation cannot be ruled out at this point, and the off-path deviation tests in Section 5.5 were designed to falsify either story.

6.4 A simple model of the async-rescue mechanism

The empirical pattern (large positive β_{LT}, smaller positive $\beta_{\sigma T}$) is striking enough that it merits a more explicit mechanism argument rather than interpretation alone. The argument that follows is not a formal theorem but a logical decomposition that derives the qualitative pattern from the structure of the friction itself.

Setup. Define a rival's update event as a calendar period in which the rival's price actually changes, that is, a period in which the rival is the mover under the asynchrony schedule. Under simultaneous play ($T = 0$) every calendar period is an update event for the rival. Under T-period strict alternation each agent's update events occur every $2T$ calendar periods (with $T = 1$, every other calendar period).

Mechanism claim (informal). Under observation latency L, an agent's perception of the rival's price is stale by $L + 1$ calendar periods. In rival update events, the same observation is stale by approximately

$$\text{rival-update staleness} \approx \frac{L+1}{2T} \quad \text{(for } T \geq 1\text{, taken as 1 when this is less than 1).}$$

At $T = 0$ this collapses to $L + 1$ rival updates. At $T = 1$ it is roughly $(L + 1)/2$. The same calendar-period latency therefore corresponds to fewer rival decisions missed when the rival updates less often.

Implication 1 (latency rescue). Take a Q-learner that observes the rival's price with latency L, meaning the value it sees is the rival's price from L periods ago. Under simultaneous play, L calendar periods of staleness correspond to L missed rival decisions, because the rival has been moving every period in between. Under strict alternation, where the rival only changes price every other period, those same L calendar periods now correspond to only $L/2$ missed rival decisions. The observation is just as stale in calendar time, but it represents far fewer actual rival moves that the agent missed. A Q-learner that struggled to coordinate with a fast-moving rival on stale data can therefore do much better when the rival is moving more slowly, even though the observation delay itself has not changed. The effect should be largest when L is large, because that is when the calendar staleness was hurting the most. Empirically, $\beta_{LT} = +0.601$ is by far the largest single interaction in the factorial, exactly as this mechanism predicts.

Implication 2 (noise: weaker, but same direction). The same logic helps with noise, but more weakly. Asynchrony does not reduce the noise on any single observation. Each one still has standard deviation σ around the rival's true price. What changes is that the rival's true price now persists for two consecutive periods rather than just one. So when an agent observes the rival at period t and again at period $t + 1$, both readings are noisy estimates of the same underlying price, rather than two noisy estimates of two different prices. The cluster of observations the Q-learner accumulates for a given rival action builds up around the right value more cleanly, which acts like a mild form of denoising by repetition. Because asynchrony does not reduce the noise on any individual observation directly and only helps through this indirect repetition channel, the rescue should be weaker than for latency. $\beta_{\sigma T} = +0.245$ is roughly half the size of $\beta_{LT} = +0.601$, exactly as the mechanism predicts.

Implication 3 (the cost of asynchrony in isolation). Asynchrony has a real cost when imposed alone. Under strict alternation each agent only moves every other period, so within the same iteration budget each agent gets half as many turns to update its Q-table. Learning is slower and the converged strategies are less polished. That cost shows up in the single-friction sweep, where Δ drops from 0.848 to 0.378 at $T = 1$. The rescue mechanism therefore involves a trade-off. With async as the only friction, the slow-learning cost dominates and Δ falls. With latency or noise also present, the benefit of having the rival's price be less out of date (because it now persists longer between updates) outweighs the slow-learning cost, and Δ ends up higher than the single-friction values would predict.

Why the off-path test is consistent. If the rescue mechanism is genuinely about the rival's perceived state being less out of date, then a forced deviation should still trigger learnt punishment behaviour in the rescue cells, because the learning is happening on real rival information rather than on a focal point lock-in. This is what Section 5.5 finds. Retaliation is present in `F101` and `F011` and recovers within 5–15 periods, and in `F111` on a longer timescale of roughly 60–75 periods consistent with the compounding effect of noise on top of the rival staleness reduction. None of the rescue cells shows the spurious focal point snap back that would be expected if the high Δ were sustained by mere convergence inertia.

What this does not establish. The mechanism claim above is an informal argument, not a derivation of the equilibrium Δ from first principles. A fully formal model would have to specify the Q-learning dynamics and the joint state evolution under combined frictions explicitly and either solve for the stationary distribution analytically (out of scope for tabular Q-learning at this state space size) or characterise it through stochastic approximation arguments. What the argument does establish is that the empirical pattern observed in the factorial and the heatmaps is consistent with a single, simple mechanism rather than a coincidence of several independent effects. Future work formalising this argument into a proper theoretical model is a natural extension.

6.5 Relation to the existing literature

The strength of the existing simulation literature is control. Calvano et al. (2020) show cleanly that independent Q-learning agents can reach cartel-like outcomes without communication, and Calvano et al. (2023) provide an off-path diagnostic for whether those outcomes are supported by punishment. The weakness, for the question asked here, is that the baseline environment is deliberately clean: rival prices are observed without delay or measurement error and both agents move on the same clock. It is therefore an ideal laboratory for proving possibility, but less suited to deciding whether real-market information imperfections dampen or preserve collusion.

The one-friction extensions each answer a narrower question well. Calvano et al. (2021) and Zhang (2025) show that imperfect monitoring and noise can weaken learned collusion, while Klein (2021) shows that alternating price setting can still sustain supracompetitive outcomes in a sequential-pricing environment. Their strength is internal clarity: each isolates one channel. Their weakness is also that isolation. A policymaker cannot infer the effect of a delayed disclosure rule, a noisy public feed, and a pricing-window rule by adding up three separate one-friction results, because Section 5 shows that those frictions interact non-linearly.

The empirical papers have the opposite trade-off. Assad et al. (2024) and Brown and MacKay (2023) study real markets, so their external validity is stronger than any laboratory simulation. They also provide the best evidence that algorithmic pricing and asynchronous update timing matter outside the model. But observational data cannot easily switch one friction on while holding the others fixed, and it cannot directly observe the counterfactual policy combinations studied here. The contribution of this thesis is to connect those two literatures: it keeps the control of the simulation setting, introduces frictions that correspond to real information and timing constraints, and shows that combinations of those frictions can reverse the conclusion one would draw from single-friction experiments. The result is not merely a robustness check on Calvano-style collusion; it identifies a new interaction mechanism that changes how policy instruments should be evaluated.

6.6 Do these frictions already exist in real markets?

You might object at this point that the three frictions tested here are not really new constraints to layer onto a Calvano-style market. They describe the conditions of any real market already. Firms observe their competitors through scraping or distribution channels with measurable latency, work from noisy data feeds, and operate on price-update cadences that vary across firms. If single information frictions individually disrupt learned collusion (and the single-friction sweeps show they do), and if those frictions are already present in any actual market, the worry about algorithmic collusion should be substantially smaller than Calvano's clean baseline suggests.

The combined results partly support and partly reject that objection. They support it in the sense that frictions do lower collusion relative to the clean baseline: moving from $\Delta = 0.848$ in `F000` to $\Delta = 0.537$ in `F111` is a reduction of about 0.31 on the Nash-to-monopoly profit scale. That is not trivial. But $\Delta = 0.537$ is still economically high: the algorithms recover more than half of the profit gap between the competitive Nash benchmark and the joint-monopoly benchmark. It is also far above the latency-plus-noise cell `F110`, where $\Delta = 0.059$. Thus the realistic joint-friction case is not as collusive as the clean Calvano baseline, but it is much more collusive than a one-friction or two-information-friction reading would suggest.

The reason is that real markets do not face one friction at a time. They face something closer to a joint cell such as `F111`. The async-rescue mechanism means that the very combination of frictions a real market has can preserve learned collusion rather than simply destroy it. Reasoning from any one friction in isolation therefore gives the wrong prediction for what happens when all three are present together.

The empirical record so far is consistent with this reading. Brown and MacKay (2023) document that firms running pricing algorithms with asymmetric update cadences in online retail sustain higher equilibrium prices than firms updating in lockstep. The signature matches the async-rescue cell of the factorial, not the single-friction collapse. So the empirical evidence available does not support a "real-world frictions prevent algorithmic collusion" inference.

The right summary is therefore not "real-world frictions prevent algorithmic collusion." It is that single information frictions disrupt learned collusion when imposed in isolation, but the way real-market frictions combine can support it. This is the central contribution of the thesis: it shows that realism does not enter the model as a simple dampening force. Some realistic frictions weaken collusion, but realistic combinations can also create new coordination channels.

6.7 Policy implication

Zhang (2025) argues for noise injection as a regulatory instrument against algorithmic collusion. The single-friction noise sweep is broadly consistent with that proposal, since a sufficiently large noise level does drive learned collusion to Nash. The combined-friction results qualify the case in two ways that matter for concrete policy design.

First, noise injection becomes substantially more effective when imposed alongside an observation delay rule. $\Delta_{\texttt{F110}} = 0.059$ versus $\Delta_{\texttt{F010}} = 0.500$. The two information degradation rules are policy complements. A regulator who can impose both should expect more disruption than the additive or multiplicative combination would predict, because the rules amplify each other.

Second, structural timing rules (mandated alternation, mandated staggered pricing windows) are policy substitutes for information degradation rules. Imposing the timing rule undoes part of the information rule's effect on learned collusion. A regulator combining "you must publish prices with bounded noise" with "you must commit to prices for a fixed window" should expect less disruption than from the noise rule alone, because the commitment window rule provides exactly the kind of structural stability that lets agents coordinate around corrupted signals.

To make this concrete, consider three real-world policy scenarios.

- **Mandated reporting delay in petrol retailing.** Germany provides a concrete example of a public fuel-price reporting system. Since 31 August 2013, petrol station operators and firms with price-setting authority have been required to report price changes for Super E5, Super E10, and Diesel in real time to the Market Transparency Unit for Fuels, which then passes the data to consumer information services (Bundeskartellamt, 2026). A rule delaying competitor access to that official feed would be the policy analogue of $L \geq 1$. The important caveat is enforcement. If firms can simply buy or build a private real-time database through scraping, mystery shopping, or direct commercial data sharing, the public-feed delay is bypassed. A reporting-delay instrument would therefore need access rules or auditing of authorised data services; otherwise it delays only one information channel, not rival observation itself.

- **Noise injection on published prices.** Zhang style mandated obfuscation of published competitor facing price feeds, in markets where firms scrape each other rather than observing directly. Effective on its own, more effective when combined with the delay above. The political and operational reality of this instrument is admittedly awkward. Regulators are not in the habit of mandating that firms publish noisy or distorted information, and a rule that essentially asks "make your prices a bit harder for competitors to read accurately" sits uncomfortably alongside the usual regulatory preference for transparency. So although the simulation results say this is the second most disruptive single instrument available, implementing it would require a regulator willing to deliberately degrade signal quality in a price-sensitive market, which is a stretch from current practice.

- **Mandated price commitment windows.** Requiring firms to commit to a price for, say, a 24 hour window before changing it. This is the analogue of $T \geq 1$. Our results suggest this disrupts collusion when imposed in isolation, but should not be combined with the previous two rules. Doing so partially undoes their effect by giving algorithms exactly the structural stability they need to coordinate around corrupted signals.

The bottom line policy lesson is that combinations of information and timing rules need to be evaluated jointly, not as the sum or product of single-instrument analyses. Combined frictions can still reduce collusion relative to the clean baseline: `F111` has $\Delta = 0.537$, well below the baseline $\Delta = 0.848$. But the reduction is much smaller than one would infer from the latency-plus-noise cell alone, where $\Delta = 0.059$. A policy designer who treats the instruments as mechanically additive would therefore over-estimate the disruption achieved by a package that also includes a timing rule.

7 Limitations and next steps

The design of this thesis is deliberately controlled: it keeps the Calvano pricing environment fixed and varies only the information and timing frictions. That is the source of the main contribution, because it makes the interaction effects interpretable. It also sets the boundary of what the results can claim. The thesis identifies mechanisms inside a stylised Q-learning environment; it does not provide a market-by-market quantitative forecast.

- **Asymmetric and heterogeneous frictions.** All three frictions are imposed symmetrically on the two agents. Real markets often feature asymmetric frictions, for example one firm with a faster price-update cadence, cleaner data feed, or stronger scraping infrastructure than the other. Brown and MacKay (2023)'s empirical findings on asymmetric update frequencies make this the most natural next extension.

- **A homogeneous two-firm product environment.** The model keeps the two firms symmetric in appeal and cost, with differentiation entering through the logit demand system. This is useful for isolating the learning mechanism, but it leaves out richer market structure: more firms, asymmetric costs, product-level heterogeneity, capacity constraints, and consumer search frictions.
- **Bounded memory and richer algorithms.** Appendix B explains why extending tabular Q-learning from one-period memory to two-period memory is computationally demanding in this state space. A meaningful extension would require either substantially larger compute, for example a high-performance-computing allocation, or a methodological switch to function-approximated Q. That step would also allow comparison with more modern pricing algorithms, but at the cost of giving up the transparent tabular mechanism used here.

These limitations do not undermine the main result. They point to where the mechanism should be stress-tested next: asymmetric firms, richer product environments, and learning rules with more memory. The central finding remains that information and timing frictions cannot be evaluated one at a time, because their combination can qualitatively change the degree of learned collusion.

References

J. Asker, C. Fershtman, and A. Pakes. The Impact of Artificial Intelligence Design on Pricing. *Journal of Economics & Management Strategy*, 33(2):276–304, 2024. doi: 10.1111/jems.12516.

S. Assad, R. Clark, D. Ershov, and L. Xu. Algorithmic Pricing and Competition: Empirical Evidence from the German Retail Gasoline Market. *Journal of Political Economy*, 132(3): 723–771, 2024. doi: 10.1086/726906.

Z. Y. Brown and A. MacKay. Competition in Pricing Algorithms. *American Economic Journal: Microeconomics*, 15(2):109–156, 2023. doi: 10.1257/mic.20210158.

Bundeskartellamt. Market Transparency Unit for Fuels. https://www.bundeskartellamt.de/EN/Tasks/markettransparencyunit_fuels/markettransparencyunit_fuels_node.html, 2026. Accessed 3 June 2026.

E. Calvano, G. Calzolari, V. Denicolò, and S. Pastorello. Artificial Intelligence, Algorithmic Pricing, and Collusion. *American Economic Review*, 110(10):3267–3297, 2020. doi: 10.1257/aer.20190623.

E. Calvano, G. Calzolari, V. Denicolò, and S. Pastorello. Algorithmic Collusion with Imperfect Monitoring. *International Journal of Industrial Organization*, 79:102712, 2021. doi: 10.1016/j.ijindorg.2021.102712.

E. Calvano, G. Calzolari, V. Denicolò, and S. Pastorello. Algorithmic Collusion: Genuine or Spurious? *International Journal of Industrial Organization*, 90:102973, 2023. doi: 10.1016/j.ijindorg.2023.102973.

Government of the Netherlands. Cartels and Competition: Business.gov.nl. https://business.gov.nl/regulations/cartels-competition/, 2024. Accessed via the Dutch government business portal.

E. J. Green and R. H. Porter. Noncooperative Collusion under Imperfect Price Information. *Econometrica*, 52(1):87–100, 1984. doi: 10.2307/1911462.

T. Klein. Autonomous Algorithmic Collusion: Q-Learning under Sequential Pricing. *RAND Journal of Economics*, 52(3):538–558, 2021. doi: 10.1111/1756-2171.12383.

E. Maskin and J. Tirole. A Theory of Dynamic Oligopoly, II: Price Competition, Kinked Demand Curves, and Edgeworth Cycles. *Econometrica*, 56(3):571–599, 1988. doi: 10.2307/1911701.

OECD. Algorithms and Collusion: Competition Policy in the Digital Age. https://www.oecd.org/competition/algorithms-collusion-competition-policy-in-the-digital-age.htm, 2017.

OECD. Algorithmic Pricing and Competition in G7 Jurisdictions: Emerging Trends and Responses. https://www.oecd.org/content/dam/oecd/en/publications/reports/2025/10/algorithmic-pricing-and-competition-in-g7-jurisdictions_f936689b/f36dacf8-en.pdf, 2025.

C. J. C. H. Watkins and P. Dayan. Q-Learning. *Machine Learning*, 8(3):279–292, 1992. doi: 10.1007/BF00992698.

N. Zhang. Too Noisy to Collude? Algorithmic Collusion Under Laplacian Noise. https://arxiv.org/abs/2509.02800, 2025.

A Implementation and design choices

The C++ simulator is structured around five executables sharing a common library: `ogcode_cpp.exe` (baseline), `ogcode_latency.exe`, `ogcode_noise.exe`, `ogcode_async.exe`, and `ogcode_combined.exe` (unified). The single-friction binaries exist primarily to cross-validate the unified one. At all-zero friction inputs the unified learner reproduces the baseline bit-for-bit, the principal correctness check.

The Calvano-style `Ran2` generator is preserved for exploration and Q-initialisation tie-breaking. Gaussian noise (Friction 2) uses an independent `std::mt19937_64` stream seeded from the session index, at $\sigma = 0$ the noise stream is short-circuited and never advanced, so the baseline is reproduced exactly when noise is off.

Price grid construction. The 15-point price grid is anchored at the two static benchmarks of the one-shot game, the symmetric Bertrand-Nash price p^{N} and the symmetric joint-monopoly price p^{M}, both computed from the demand and cost parameters before any learning runs. The grid extends a fraction $\xi = 0.1$ beyond each benchmark so that Nash and monopoly are interior points of the action set:

$$p_1 = p^{\mathrm{N}} - \xi(p^{\mathrm{M}} - p^{\mathrm{N}}), \qquad p_m = p^{\mathrm{M}} + \xi(p^{\mathrm{M}} - p^{\mathrm{N}}), \qquad p_k = p_1 + \tfrac{k-1}{m-1}(p_m - p_1). \tag{3}$$

For the baseline parameters this gives $p_1 \approx 1.428$ and $p_{15} \approx 1.970$.

Grid robustness. The baseline result is not a coarse-grid artefact. With Calvano's original fixed-β protocol, Δ falls modestly with finer grids and asymptotes by $m \sim 50$:

m	sessions	Δ	SD	avg time-to-convergence
15	1000	0.848	0.109	261 episodes
25	1000	0.806	0.076	364 episodes
50	250	0.707	0.023	476 episodes
100	250	0.706	0.016	1144 episodes

A ν corrected version (in which β is scaled with m to hold expected per-cell visit count constant per Equation 4) is left for future work. Without it, some of the Δ decline from $m = 15$ to $m = 100$ may reflect worse exploration per cell rather than a genuine effect of grid resolution.

Replication-count stability for combined-friction cells. Figure 10 plots the combined-friction cell means at $N \in \{250, 500, 1000\}$ sessions. It is a visual version of Table 2. The ordering of cells and the interaction pattern are stable across replication counts; the largest movement between $N = 250$ and $N = 1000$ is about 0.023 in Δ.

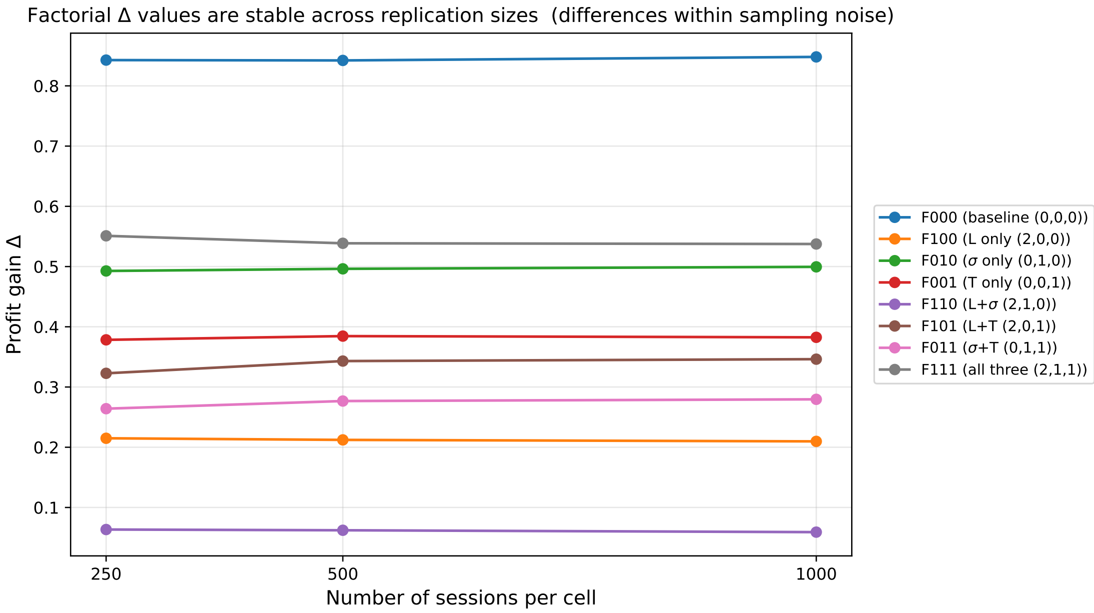

Figure 10: Combined-friction Δ values across replication sizes. The pattern is qualitatively identical at $N \in \{250, 500, 1000\}$, with cell-level movements between consecutive N's typically under 0.025.

B Why bounded memory was not extended

A natural extension of the Calvano baseline is to let algorithms remember more than the immediately previous period. This thesis keeps memory fixed at one period and instead studies information and timing frictions on the rival price observation. The reason is computational, and it is worth stating quantitatively because the size of the obstacle is not obvious from a casual read.

Each agent's state is the joint price vector over the previous k periods, so the state space has cardinality $|\mathcal{S}| = m^{nk}$ and the per-agent Q-table has m^{nk+1} cells. The Calvano baseline uses $(n, k, m) = (2, 1, 15)$, giving $|\mathcal{S}| = 225$ and 3,375 cells per agent, which is a manageable problem. Calvano et al. (2020, footnote to eq. 21) give the expected number of visits to a sub-optimal Q-cell under ε-greedy exploration as

$$\nu = \frac{(m-1)^n}{m^{kn(n+1)} \cdot \left(1 - e^{-\beta(n+1)}\right)}. \tag{4}$$

Holding β and the iteration budget fixed and incrementing k from 1 to 2 with $n = 2$, $m = 15$ multiplies the denominator by $m^{n(n+1)} = 15^6 \approx 1.14 \times 10^7$. In plain English, doubling memory makes each Q-cell receive about ten million times fewer learning updates within the same compute budget. To compensate would require either a 10^7-fold longer iteration budget per session (pushing session runtimes from minutes to weeks), or a correspondingly larger exploration parameter (which would destroy the late session low-ε exploitation regime needed for collusion to crystallise). Neither is feasible at the available compute.

The implication is that any meaningful bounded memory extension at this Calvano style tabular Q baseline would require either substantially larger compute (a proper HPC allocation), or a methodological change to function approximated Q that abandons the per-cell visit count argument entirely. Both are reasonable next steps but were out of scope for this thesis. The frictions that were implemented, namely latency, noise, and asynchrony, all preserve the per-agent state cardinality at $m^2 = 225$, which is exactly why they are tractable on the same compute that runs the baseline.